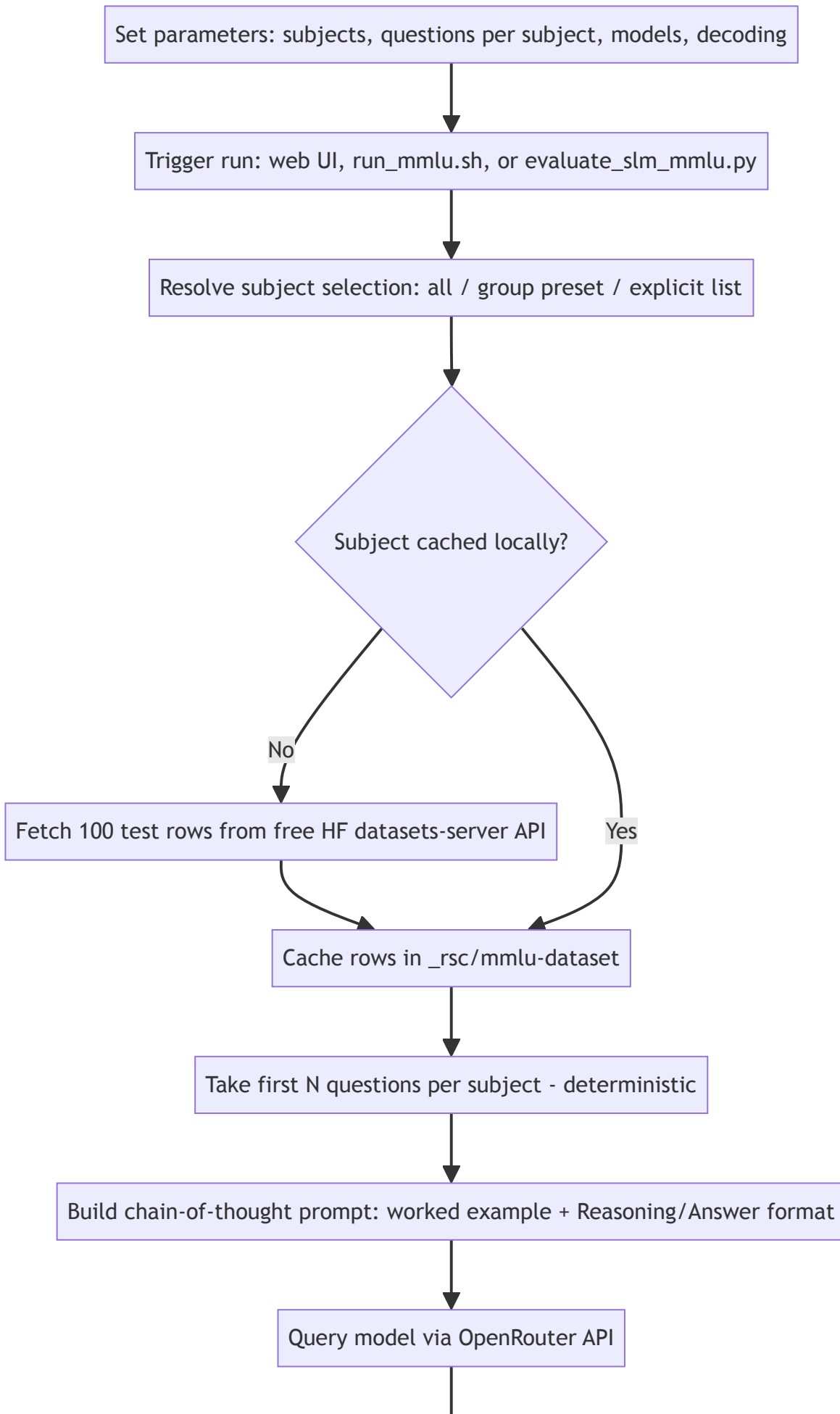
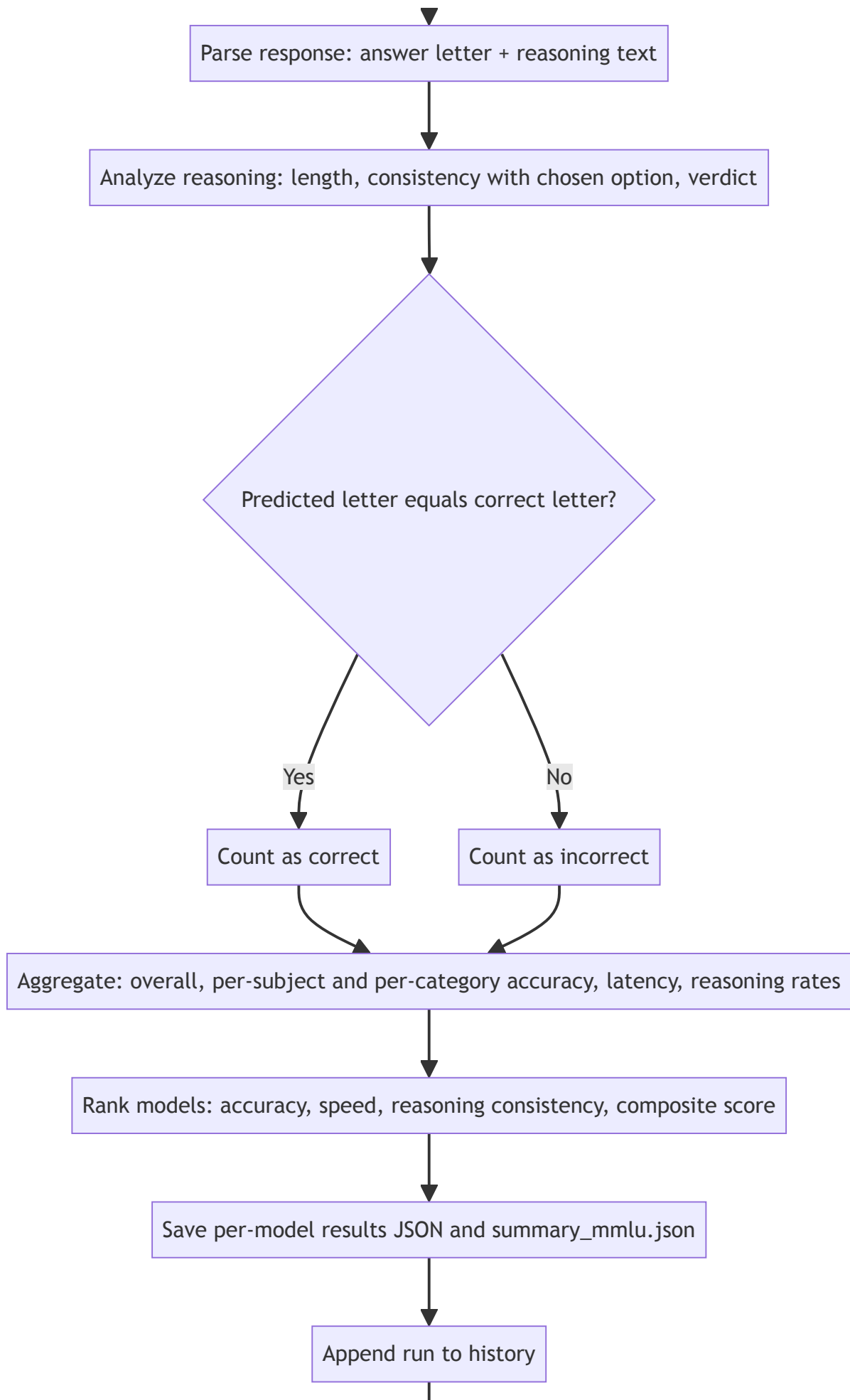


MMLU Process Flow

How the MMLU benchmark runs in this project, from a click on "Run MMLU" (or `./run_mmlu.sh`) to the ranking table, charts, and per-question reasoning shown in the web app. MMLU (Massive Multitask Language Understanding) is a multiple-choice knowledge and reasoning test spanning 57 subjects grouped into four categories (STEM, Humanities, Social Sciences, Other). For every question the model must first write short step-by-step reasoning and then commit to one of four options (A-D); both the final letter and the reasoning text are parsed and evaluated.

Workflow diagram







Web app: ranking table, charts, and per-question Q/A + reasoning viewer

Chronological steps

#	Step	What happens	Where (file / function)
1	Parameters	Read subjects, questions per subject, models, decoding params (all documented at the top of the script and shell runner)	<code>evaluate_slm_mmlu.py</code> (RUN PARAMETERS), <code>run_mmlu.sh</code>
2	Trigger	User clicks Run MMLU, runs <code>./run_mmlu.sh</code> , or <code>python evaluate_slm_mmlu.py</code>	<code>passenger_wsgi.py</code> (<code>/mmlu/run</code>) or <code>run_mmlu_evaluation</code>
3	Resolve subjects	Turn "all", a group preset, or a list into valid subject names	<code>resolve_subjects</code>
4	Fetch questions	Download the first 100 test rows per subject from the free Hugging Face datasets-server API (cais/mmlu, fallback tasksource/mmlu); no API key needed	<code>_download_subject</code>
5	Cache	Store rows in <code>_rsc/mmlu-dataset/<subject>.json</code> so reruns are offline and repeatable	<code>fetch_subject_questions</code>
6	Sample	Take the first N questions per subject (deterministic, no randomness)	<code>fetch_subject_questions</code> , <code>load_mmlu_tasks</code>
7	Build prompt	Chain-of-thought prompt: one worked example, strict Reasoning: then Answer: <letter> output format	<code>build_mmlu_prompt</code> , <code>WORKED_EXAMPLE</code>
8	Query model	Send the prompt to the chosen model through OpenRouter	<code>query_model_mmlu</code> (HTTP POST)
9	Parse	Extract the answer letter AND the reasoning text (handles Reasoning:/Answer:, <think> blocks, "the answer is (B)", bare letters, truncation)	<code>parse_mmlu_response</code>
10	Evaluate reasoning	Score the reasoning: is it present, how long, does it actually support the chosen option; assign a verdict	<code>analyze_reasoning</code>
11	Compare	Exact match: predicted letter equals the correct letter	<code>evaluate_model_mmlu</code>
12	Aggregate	Overall / per-subject / per-category accuracy, latency, errors, reasoning rates	<code>evaluate_model_mmlu</code>
13	Rank	Per-dimension ranks and composite score across all evaluated models	<code>build_mmlu_summary</code>
14	Save	Write <code>results/<model>_mmlu.json</code> and <code>results/summary_mmlu.json</code>	<code>run_mmlu_evaluation</code> / <code>run_mmlu_benchmark</code>

#	Step	What happens	Where (file / function)
15	History	Append the run (models, subjects, question count, params) to history	<code>passenger_wsgi.append_history</code>
16	Present	Ranking table, accuracy charts, and the per-question Q/A + reasoning accordion	<code>templates/mmlu.html</code> (Chart.js)

Components

Component	File	Role
Run parameters	<code>evaluate_slm_mmlu.py</code> (top), <code>run_mmlu.sh</code> (top)	Subject selection ("all", group presets, or explicit list), questions per subject (1-100), models, decoding params - every option listed in comments
Subject catalogue	<code>MMLU_SUBJECTS</code> , <code>SUBJECT_GROUPS</code> , <code>CATEGORY_LABELS</code>	The 57 subjects mapped to their official categories; group presets derived from the mapping
Dataset fetcher	<code>fetch_subject_questions</code> , <code>_download_subject</code>	Free HF datasets-server API client with source fallback and local JSON cache
Task loader	<code>load_mmlu_tasks</code>	Flattens subjects x questions into one ordered task list
Prompt builder	<code>build_mmlu_prompt</code> , <code>WORKED_EXAMPLE</code>	Chain-of-thought prompt with a worked example enforcing a parseable format
Model client	<code>query_model_mmlu</code>	OpenRouter chat-completions call with timing and error capture
Response parser	<code>parse_mmlu_response</code>	Splits a raw response into (answer letter, reasoning text)
Reasoning analyzer	<code>analyze_reasoning</code>	Heuristic quality check: presence, word count, consistency, verdict
Evaluator	<code>evaluate_model_mmlu</code>	Runs all tasks for one model; aggregates accuracy and reasoning metrics
Ranker	<code>build_mmlu_summary</code>	Per-dimension ranks + composite score across models
CLI runner	<code>run_mmlu_evaluation</code> , <code>_parse_cli</code> , <code>run_mmlu.sh</code>	One-shot terminal pipeline: setup, install, run, print ranking
Web routes	<code>passenger_wsgi.py</code> : <code>/mmlu</code> , <code>/mmlu/run</code> , <code>/mmlu/metrics</code> , <code>/mmlu/details</code>	Online runs, ranking JSON, and the per-question detail feed
Web page	<code>templates/mmlu.html</code>	Subject picker with presets, decoding sliders, ranking table, charts, Q/A + reasoning viewer

Models evaluated

Model	Developer	Params	OpenRouter id	Signature technique
Gemma-3-4B	Google	4B	google/gemma-3-4b-it	Knowledge distillation + local/global attention mix
Llama-3.2-3B	Meta	3B	meta-llama/llama-3.2-3b-instruct	Compact dense transformer, pruned/distilled from 3.1
Ministral-8B	Mistral AI	8B	mistralai/ministral-8b-2512	Sliding window attention + GQA

Gemma-3-4B replaced Phi-4-Mini, which no longer has any active endpoints on OpenRouter (every request returned 404 and scored 0%).

Metrics produced

Metric	Definition
Accuracy	correct letters / total questions (exact match)
Category accuracy	Accuracy aggregated over STEM / Humanities / Social Sciences / Other
Subject accuracy	correct / total per individual subject
Average response time	Mean wall-clock seconds per API call
Errors	Count of failed or timed-out API calls
Reasoning rate	Share of answers that came with a non-trivial explanation (≥ 5 words)
Reasoning consistency	Share of answers whose reasoning actually supports the chosen option (names the letter or reuses the choice's key words)
Avg reasoning words	Mean length of the parsed reasoning text
Composite score	$0.70 \times \text{accuracy} + 0.15 \times \text{reasoning consistency} + 0.15 \times \text{relative speed}$ (fastest avg time / model's avg time)

Reasoning verdicts (per question)

Verdict	Meaning
sound	Correct answer, and the reasoning clearly supports it
right-weak-link	Correct answer, reasoning present but does not clearly support the pick
lucky-guess	Correct answer with no real reasoning
flawed-reasoning	Reasoned its way to a wrong answer
blind-guess	Wrong answer and no reasoning
no-answer	No A-D letter could be parsed (API error or malformed output)

Ranking

Models are ranked per dimension (1 = best) on accuracy, speed, and reasoning consistency, then ordered by the composite score. Accuracy dominates the composite (70%); reasoning quality and latency (15% each) break ties, which mirrors how small language models are picked in practice: quality first, then cost and latency.

ELI5: Give the model a school quiz across 57 subjects. For every question it must "show its work" before circling A, B, C, or D. We check the circled letter against the answer key, read the shown work to see if it actually justifies the pick, time every answer, and then rank the models like a report card: mostly on how many they got right, with bonus points for honest working-out and for being quick.